Reducing Inter-Component Relaxations for the All-Pairs Shortest Path Problem

Dani Zugan[1][0009−0006−8819−2538], Rok Požar[2][0000−0002−2037−9535], and Andrej Brodnik[1,2][0000−0001−9773−0664]

[1] University of Ljubljana, Faculty of Computer and Information Science, Slovenia
[2] University of Primorska, Faculty of Mathematics, Natural Sciences and Information Technologies, Slovenia

Abstract. The all-pairs shortest path problem is a fundamental problem in graph algorithms, with applications in telecommunications, logistics, and network analysis. In this paper, we study this problem in directed graphs and introduce a new relaxation-based approach. The method decomposes the graph into strongly connected components, computes shortest paths within each component, and then determines shortest paths between different components. Unlike earlier component-based methods that propagate distances directly through outgoing edges, the proposed approach uses a two-phase tail-head relaxation scheme. Here, a tail is the starting vertex of an outgoing edge of a component, and a head is its ending vertex outside that component. In the first phase, distances are propagated through these vertices; in the second phase, the resulting values are extended through subsequent components. We prove that this decomposition correctly computes inter-component shortest-path distances once shortest paths within the relevant subgraphs are known. We also derive the running-time bound of the algorithm. In the worst case, it matches the cubic complexity of the classical Floyd–Warshall algorithm, while for graphs with small sets of tail and head vertices it can substantially reduce the number of required relaxations.

Keywords: all-pairs shortest path · strongly connected components · Floyd-Warshall algorithm.

1 Introduction

Many real-world problems in bioinformatics, logistics, telecommunications, and related domains are modeled as graphs, where the main task is to find shortest paths between pairs of nodes [1]. The shortest path problem appears in two variants: finding shortest paths from a single source to all other nodes (single-source shortest path, SSSP), and finding shortest paths for all node pairs (all-pairs shortest path, APSP).

Two conceptually different strategies exist for APSP. The first reduces APSP to repeated SSSP computations—running an SSSP algorithm from every node and aggregating the results. The second constructs shortest paths for all pairs

simultaneously. For graphs with non-negative edge weights, the repeated SSSP strategy uses Dijkstra-type algorithms.

A classical representative of the simultaneous strategy is the Floyd–Warshall algorithm [10, 19], based on dynamic programming. Despite their different viewpoints, both strategies rely on the same operation: *relaxation*. Since the overall running time depends on the number of relaxation attempts, repeated SSSP approaches tend to be preferable on sparse graphs, whereas Floyd–Warshall-like methods suit dense graphs.

The SCC-based framework used in this paper originates from [20]. In that work, the graph is first decomposed into strongly connected components (SCCs), and APSP is solved independently within each SCC, which is possible because all vertex pairs inside an SCC are mutually reachable.

The remaining task is to compute shortest paths between vertices that belong to different components. In [20], this is done by propagating distances across components through outgoing edges, using a reachability observation: if an arbitrarily chosen vertex in a source component cannot reach an arbitrarily chosen vertex in a destination component, then relaxations between these two components can be omitted. Our work adopts the same SCC decomposition and the same reachability viewpoint, but replaces the edge-based inter-component propagation of [20] with a computation based on classical Floyd–Warshall relaxations.

The goal of this paper is to introduce an SCC-guided approach to the APSP problem, where relaxations are performed only over relevant vertex triples, and to evaluate its empirical performance against established algorithms. The proposed method is tested on synthetic graphs generated by the Erdős–Rényi model [8, 9] and the Barabási–Albert model [3], as well as on real-world graphs derived from practical datasets.

Section 2 surveys related work on APSP algorithms and Section 3 introduces the necessary definitions. Our algorithm is presented in Section 4. Section 5 concludes with a summary of results and insights.

2 Related Work

The all-pairs shortest path (APSP) problem has been studied extensively in the graph algorithms literature. The classical dynamic-programming solution is the Floyd–Warshall algorithm [10, 19], which runs in $O(n^3)$ time. A different approach is to solve APSP through repeated single-source shortest path computations. For graphs with non-negative edge weights, this is typically done with Dijkstra's algorithm [7], while Johnson's algorithm [13] extends this approach to graphs without negative cycles.

Several improvements of Floyd–Warshall aim to reduce the number of unsuccessful relaxations. In [4], the authors introduce the Tree and Hourglass data structures and obtain an expected running time of $O(n^2 \log^2 n)$ for complete directed graphs with edge weights drawn uniformly from $[0, 1]$. Similarly, [15] proposes the SmartForce technique and a hybrid variant that processes only selected relaxations likely to improve distances.

Other work exploits sparsity or structural restrictions. The Rectangular Algorithm [2] skips inactive regions of the distance matrix, while [18] improves Floyd–Warshall on disconnected and very sparse graphs by maintaining reachability information.

A closely related line of work is based on strongly connected components (SCCs). SCC decomposition can be computed in linear time by Tarjan's algorithm [16], and its structural depth-first-search foundations are revisited in [17]. Building on this idea, [20] first solves shortest paths inside each component and then propagates distances between components by relaxing via outgoing edges from components.

For sparse graphs, repeated single-source methods remain a strong baseline. Their performance depends heavily on the priority queue used inside Dijkstra's algorithm. Fibonacci heaps give the classical theoretical bound [12], while pairing heaps are often competitive in practice [11].

Finally, for complete directed graphs with independent edge weights drawn uniformly from $[0, 1]$, sharper expected bounds are known. The Hidden Paths algorithm [14] achieves expected running time $O(n^2 \log n)$, and this bound was later matched by Demetrescu and Italiano, who also provided an experimental analysis of APSP algorithms [5, 6].

3 Preliminaries

A directed graph is denoted by $G = (V, E)$, where V is a finite, non-empty set of vertices and $E \subseteq V \times V$ is a set of directed edges. We write $V = \{v_1, \ldots, v_n\}$.

An edge $e = (u, v) \in E$ is directed from u to v. The vertex u is called the initial vertex of e, denoted by $\mathrm{ini}(e) = u$, while v is called the terminal vertex of e, denoted by $\mathrm{ter}(e) = v$.

A path in G is a sequence of pairwise distinct vertices $P = (v_1, \ldots, v_k)$ such that $(v_i, v_{i+1}) \in E$ for all $1 \leq i < k$. We say that v is *reachable* from u if there exists a path starting at u and ending at v.

If $P = (v_1, \ldots, v_k)$ is a path and $1 \leq a \leq b \leq k$, then $(v_a, \ldots, v_b)$ is called a *subpath* of P. When referring to such subpaths, we write $a \rightsquigarrow b$.

A graph $G' = (V', E')$ is a subgraph of G if $V' \subseteq V$ and $E' \subseteq E$.

A strongly connected component (SCC) is a maximal set of vertices $C \subseteq V$ such that every two vertices in C are mutually reachable.

A directed graph can be partitioned into t strongly connected components $C_1, C_2, \ldots, C_t$, ordered topologically such that there are no edges from C_j to C_i for $j > i$. Further, we denote the union $C_i \cup C_{i+1} \cup \cdots \cup C_t$ by $C^{(i \ldots t)}$.

Given an SCC $C_i \subseteq V$, the set of outgoing edges from C_i is defined as $\mathrm{Out}(C_i) = \{(u, v) \in E \mid u \in C_i,\ v \notin C_i\}$ and the set of incoming going edges to C_i is defined as $In(C_i) = \{(u, v) \in E \mid u \notin C_i,\ v \in C_i\}$.

For an SCC C_i, we denote by $I_i = \{ini(e) \mid e \in \mathrm{Out}(C_i)\}$ the set of initial vertices $\mathrm{Out}(C_i)$, and by $T_i = \{ter(e) \mid e \in \mathrm{Out}(C_i)\}$ the set of terminal vertices of outgoing edges by .

Note that distinct edges in $\mathrm{Out}(C_i)$ may share the same initial vertex or the same terminal vertex. Thus, multiple outgoing edges may originate at a single vertex in I_i, and multiple outgoing edges may enter the same vertex in T_i.

For vertex $v \in C^{(i+1\ldots t)}$ we define $\mathrm{In}_i(v) = \{e \in \mathrm{Out}(C_i) \mid \mathrm{ter}(e) = v\}$.

A weighted graph is a graph $G = (V, E)$ with a weight function $w\colon E \to \mathbb{R}$ that assigns each directed edge $e \in E$ a weight $w(e)$. This weight function extends to a path $P = (v_1, v_2, \ldots, v_k)$ by $w(P) = \sum_{i=1}^{k-1} w(v_i, v_{i+1})$.

A shortest path from s to d is a path of minimum weight among all paths from s to d. The distance between two vertices s and d, denoted by $D_{s,d}$, is the weight of a shortest path from s to d in G.

For a component C_i, we write $D^{(i)}_{u,v}$ for the distance from u to v restricted to paths whose vertices lie entirely in C_i. Similarly, $D^{(i\ldots t)}_{u,v}$ denotes the distance from u to v restricted to paths whose vertices lie in $C^{(i\ldots t)}$.

For simplicity, we will refer to directed graphs simply as *graphs* throughout the rest of this paper.

4 Our Solution

We propose an alternative algorithm for computing all-pairs shortest paths in graphs, building directly on the SCC-based solution of [20]. The algorithm of [20] presented in Algorithm 1 computes inter-component shortest paths by relaxing over all edges in $\mathrm{Out}(C_i)$ in lines 8-10. To estimate the size of $|\mathrm{Out}(C_i)|$ we observe that in the worst case, every vertex in C_i has an outgoing edge to every vertex in $C^{(i+1\ldots t)}$. Consequently, $|\mathrm{Out}(C_i)|$ can be as large as $O(|C_i|\cdot|C^{(i+1\ldots t)}|)$, which pushes the overall complexity to $\omega(n^3)$. In the rest of the section we design algorithm which avoids relaxation over $\mathrm{Out}(C_i)$.

Algorithm 1 APSP algorithm in [20]

```
1: for i = 1 to t do
2:     D^(i)_{s,d} ← APSP(C_i)
3: end for
4: for i = t − 1 to 1 do
5:     for each node d ∈ C^(i+1...t) do
6:         for each node s ∈ C_i do
7:             D^(i...t)_{s,d} ← ∞
8:             for each edge e ∈ Out(C_i) do
9:                 D^(i...t)_{s,d} ← min(D^(i...t)_{s,d}, D^(i)_{s,ini(e)} + w(e) + D^(i+1...t)_{ter(e),d})
10:            end for
11:        end for
12:    end for
13: end for
```

4.1 Reducing the number of Floyd–Warshall Relaxation

Every path (also an optimal one) from $s \in C_i$ and $d \in C^{(i+1\ldots t)}$ consists of three subpaths: the first one is in C_i, the second one is an edge $e \in \mathrm{Out}(C_i)$, and the third one is in $C^{(i+1\ldots t)}$. Let $t = \mathrm{ini}(e)$ and $h = \mathrm{ter}(e)$, then the path in general can be written as $(s, \ldots, t, h, \ldots, d)$, where might $s = t$ or $h = d$. Further, let C_i^+ be a graph consisting of C_i, and edges and vertices of $\mathrm{Out}(C_i)$. This brings us to the following lemma:

Lemma 1. *Let $(s, \ldots, h, \ldots, d)$ be a shortest path from $s \in C_i$ to $d \in C^{(i+1\ldots t)}$, where $h = \mathrm{ter}(e)$ for some $e \in \mathrm{Out}(C_i)$. Then $(s, \ldots, h)$ is a shortest path from s to h in C_i^+ and $(h, \ldots, d)$ is a shortest path from h to d in $C^{(i+1\ldots t)}$.*

Proof. We prove the lemma by contradiction. We define $D^{(i)+}_{s,h}$ as the length of a shortest path from s to h, where path's edges are in C_i^+.

Assume that for some $s \in C_i$ and some $d \in C^{(i+1\ldots t)}$

$$D^{(i\ldots t)}_{s,d} < {}_H D^{(i\ldots t)}_{s,d}.$$

Let P be a path from s to d with $w(P) = D^{(i\ldots t)}_{s,d}$. Then:

$$w(P) = D^{(i\ldots t)}_{s,d} < {}_H D^{(i\ldots t)}_{s,d} = \min_{h \in H_i} \left({}_T D^{(i)+}_{s,h} + D^{(i+1\ldots t)}_{h,d} \right).$$

Since $s \in C_i$ and $d \notin C_i$, the path P must leave component C_i. Therefore, there exists at least one edge $(u, v) \in \mathrm{Out}(C_i)$ along this path. The path P thus decomposes into two subpaths connected by (u, v):

$$P = s \rightsquigarrow u, v \rightsquigarrow d.$$

Since the subpaths $s \rightsquigarrow u$ and $v \rightsquigarrow d$ are no shorter than the corresponding shortest paths, we have:

$$w(P) \geq D^{(i)}_{s,u} + w(u, v) + D^{(i+1\ldots t)}_{v,d}.$$

By the definition of ${}_T D^{(i)+}_{s,v}$:

$$D^{(i)}_{s,u} + w(u, v) \geq \min_{e \in \mathrm{In}_i(h)} \left(D^{(i)}_{s,\mathrm{ini}(e)} + w(e) \right) = {}_T D^{(i)+}_{s,v},$$

and thus $w(P) \geq {}_T D^{(i)+}_{s,v} + D^{(i+1\ldots t)}_{v,d}$. Moreover, by the definition of ${}_H D^{(i\ldots t)}_{s,d}$:

$$D^{(i\ldots t)}_{s,d} = w(P) \geq {}_T D^{(i)+}_{s,v} + D^{(i+1\ldots t)}_{v,d} \geq \min_{h \in H_i} \left({}_T D^{(i)+}_{s,h} + D^{(i+1\ldots t)}_{h,d} \right) = {}_H D^{(i\ldots t)}_{s,d},$$

a contradiction.

Given distances $D^{(i)}_{s,d}$ and $D^{(i+1\ldots t)}_{s,d}$ between vertex pairs within C_i and within $C^{(i+1\ldots t)}$, respectively, we propose a two-step decomposition:

Bridge step. For each $s \in C_i$ and $h \in H_i$, let

$$ {}_T D^{(i)+}_{s,h} \;=\; \min_{e \in \mathrm{In}_i(h)} \left(D^{(i)}_{s,\mathrm{ini}(e)} + w(e) \right). $$

Reduced-relaxation step. For each $s \in C_i$ and $d \in C^{(i+1\ldots t)}$, let

$$ {}_H D^{(i\ldots t)}_{s,d} \;=\; \min_{h \in H_i} \left({}_T D^{(i)+}_{s,h} + D^{(i+1\ldots t)}_{h,d} \right). $$

Theorem 1. *With notation and assumptions above, for each $s \in C_i$ and $d \in C^{(i+1\ldots t)}$ we have $D^{(i\ldots t)}_{s,d} = {}_H D^{(i\ldots t)}_{s,d}$.*

Proof. We prove the theorem by contradiction. Assume that for some $s \in C_i$ and some $d \in C^{(i+1\ldots t)}$

$$ D^{(i\ldots t)}_{s,d} < {}_H D^{(i\ldots t)}_{s,d}. $$

Let P be a path from s to d with $w(P) = D^{(i\ldots t)}_{s,d}$. Then:

$$ w(P) = D^{(i\ldots t)}_{s,d} < {}_H D^{(i\ldots t)}_{s,d} = \min_{h \in H_i} \left({}_T D^{(i)+}_{s,h} + D^{(i+1\ldots t)}_{h,d} \right). $$

Since $s \in C_i$ and $d \notin C_i$, the path P must leave component C_i. Therefore, there exists at least one edge $(u, v) \in \mathrm{Out}(C_i)$ along this path. The path P thus decomposes into two subpaths connected by (u, v):

$$ P = s \rightsquigarrow u, v \rightsquigarrow d. $$

Since the subpaths $s \rightsquigarrow u$ and $v \rightsquigarrow d$ are no shorter than the corresponding shortest paths, we have:

$$ w(P) \geq D^{(i)}_{s,u} + w(u, v) + D^{(i+1\ldots t)}_{v,d}. $$

By the definition of ${}_T D^{(i)+}_{s,v}$:

$$ D^{(i)}_{s,u} + w(u, v) \geq \min_{e \in \mathrm{In}_i(h)} \left(D^{(i)}_{s,\mathrm{ini}(e)} + w(e) \right) = {}_T D^{(i)+}_{s,v}, $$

and thus $w(P) \geq {}_T D^{(i)+}_{s,v} + D^{(i+1\ldots t)}_{v,d}$. Moreover, by the definition of ${}_H D^{(i\ldots t)}_{s,d}$:

$$ D^{(i\ldots t)}_{s,d} = w(P) \geq {}_T D^{(i)+}_{s,v} + D^{(i+1\ldots t)}_{v,d} \geq \min_{h \in H_i} \left({}_T D^{(i)+}_{s,h} + D^{(i+1\ldots t)}_{h,d} \right) = {}_H D^{(i\ldots t)}_{s,d}, $$

a contradiction.

4.2 Algorithm

Algorithm 2 implements the above two-phase relaxation for all components, processing them in reverse topological order so that distances within $C^{(i+1\ldots t)}$ are available when component C_i is processed.

Algorithm 2 Tail-Head relaxation APSP algorithm

```
1: Tarjan(G)
2: for i = 1 to t do
3:     Solve APSP inside C_i
4: end for
5: for i = t − 1 to 1 do
6:     for each edge e ∈ Out(C_i) do                          ▷ Bridge step
7:         for each node s ∈ C_i do
8:             D_{s,h}^{(i...t)} ← min( D_{s,h}^{(i...t)}, D_{s,k}^{(i)} + w(ini(e), ter(e))^{(i...t)} )
9:         end for
10:    end for
11:    for each node k ∈ H_i do                               ▷ Reduced-relaxation step
12:        for each node s ∈ C_i do
13:            for each node d reachable from k in C^{(i+1...t)} do
14:                D_{s,d}^{(i...t)} ← min( D_{s,d}^{(i...t)}, D_{s,k}^{(i...t)} + D_{k,d}^{(i+1...t)} )
15:            end for
16:        end for
17:    end for
18: end for
```

In the reduced-relaxation step, one could in principle iterate over all vertices in $C^{(i+1\ldots t)}$. However, for a fixed head vertex $h \in H_i$, only vertices reachable from h can lead to a finite value of

$$ {}_T D_{s,h}^{(i)+} + D_{h,d}^{(i+1\ldots t)}, $$

and therefore only those vertices need to be considered.

This observation can be combined with the Tree algorithm of [4]. For each head vertex h, the predecessor information obtained during the APSP computation on $C^{(i+1\ldots t)}$ defines a shortest-path tree rooted at h containing the vertices reachable from h. Instead of relaxing all reachable vertices independently, we traverse this tree in depth-first order.

The Tree algorithm provides an additional reduction beyond reachability filtering. If relaxation through h improves the current distance to a vertex d, the traversal continues with the next vertex in DFS order. If the relaxation does not improve $D_{s,d}$, the entire subtree rooted at d can be skipped, since relaxation through h cannot improve the distance to any descendant of d in the shortest-path tree.

Thus, the tree-based reduced-relaxation step avoids both unreachable vertices and relaxation attempts that are guaranteed not to improve the current distances. In the worst case, all reachable vertices are still processed, but when unsuccessful relaxations occur near the roots of large subtrees, a substantial number of relaxation attempts can be avoided. Incorporation of tree-based reduced-relaxation step is presented in Algorithm 3.

Algorithm 3 Incorporation of Tree algorithm in Reduced-Relaxation step

```
1: Obtain π, an n × n matrix with predecessor information from the APSP computation on C^(i+1...t).
2: for each node k ∈ T_i do
3:     OUT_k ← ConstructOUT_k(π)
4:     for each node s ∈ C_i do
5:         Stack = empty
6:         Stack.push(k)
7:         while Stack ≠ empty do
8:             Stack.pop()
9:             for all children j of x in OUT_k do
10:                if D_{s,k} + D_{k,j} < D_{s,j} then
11:                    D_{s,j} = D_{s,k} + D_{k,j}
12:                    π_{s,j} = π_{kj}
13:                end if
14:            end for
15:        end while
16:    end for
17: end for
```

4.3 Time Complexity

Computing SCCs with Tarjan's algorithm takes $O(|V|+|E|)$ time. Solving APSP within each component C_i takes $O(|C_i|^3)$ time. Having shortest paths inside the component already computed, for the bridge step, the algorithm considers every source $s \in C_i$ and every outgoing edge $(k, h) \in \mathrm{Out}(C_i)$, resulting in $O(|C_i| \cdot |\mathrm{Out}(C_i)|)$ relaxations. The reduced-relaxation step performs at most $O(|H_i| \cdot |C_i| \cdot |C^{(i+1\ldots t)}|)$ relaxations. Incorporating the Tree algorithm does not change this asymptotic bound, since in the worst case all vertices in $C^{(i+1\ldots t)}$ may still need to be processed.

Summing the computations within and between all components, the total running time is

$$O\left(\sum_{i=1}^{t} |C_i|^3 + \sum_{i=1}^{t-1} |C_i| \cdot \left(|\mathrm{Out}(C_i)| + |H_i| \cdot |C^{(i+1\ldots t)}|\right)\right). \tag{1}$$

In the worst case, this is $O(n^3)$, matching the classical Floyd–Warshall algorithm. For strongly disconnected graphs where the components, the sets H_i, and the sets of outgoing edges are small, the number of inter-component relaxations is substantially reduced.

References

1. Ahuja, R.K., Magnanti, T.L., Orlin, J.B.: Network flows: Theory, Algorithms and Applications. Prentice-Hall, Inc. (1993)

2. Aini, A., Salehipour, A.: Speeding up the floyd–warshall algorithm for the cycled shortest path problem. Applied Mathematics Letters **25**, 1–5 (2012)
3. Barabási, A., Albert, R.: Emergence of scaling in random networks. Science **286**, 509–512 (1999)
4. Brodnik, A., Grgurovič, M., Požar, R.: Modifications of the floyd–warshall algorithm with nearly quadratic expected-time. Ars Mathematica Contemporanea **22**, 1–22 (2021)
5. Demetrescu, C., Italiano, G.F.: A new approach to dynamic all pairs shortest paths. Journal of the ACM **51**, 968–992 (2004)
6. Demetrescu, C., Italiano, G.F.: Experimental analysis of dynamic all pairs shortest path algorithms. ACM Transactions on Algorithms **2**, 578–601 (2006)
7. Dijkstra, E.W.: A note on two problems in connexion with graphs. Numerische Mathematik **1**, 269–271 (1959)
8. Durrett, R.: Random Graph Dynamics, Cambridge Series in Statistical and Probabilistic Mathematics, vol. 5. Cambridge University Press (2006)
9. Erdős, P., Rényi, A.: On random graphs i. Publicationes Mathematicae **6**, 290–297 (1959)
10. Floyd, R.W.: Algorithm 97: Shortest path. Communications of the ACM **5**, 345 (1962)
11. Fredman, M.L., Sedgewick, R., Sleator, D.D., Tarjan, R.E.: The pairing heap: A new form of self-adjusting heap. Algorithmica **1**(1), 111–129 (1986)
12. Fredman, M.L., Tarjan, R.: Fibonacci heaps and their uses in improved network optimization algorithms. J. ACM **34**(3), 596–615 (July 1987). https://doi.org/10.1145/28869.28874, https://doi.org/10.1145/28869.28874
13. Johnson, D.B.: Efficient algorithms for shortest paths in sparse networks. Journal of the ACM **24**, 1–13 (1977)
14. Karger, D., Koller, D., Phillips, S.J.: Finding the hidden path: time bounds for all-pairs shortest paths. SIAM Journal on Computing **22**, 1199–1217 (1993)
15. Lancia, G., Dalpasso, M.: Speeding up floyd–warshall's algorithm to compute all-pairs shortest paths and the transitive closure of a graph. Algorithms **18**(9), 560 (2025)
16. Tarjan, R.E.: Depth-first search and linear graph algorithms. SIAM Journal on Computing **1**(2), 146–160 (1972)
17. Tarjan, R.E., Zwick, U.: Finding strong components using depth-first search. European Journal of Combinatorics **119** (2024)
18. Toroslu, I.H.: The floyd–warshall all-pairs shortest paths algorithm for disconnected and very sparse graphs. Software: Practice and Experience **53**, 1287–1303 (2023)
19. Warshall, S.: A theorem on boolean matrices. Journal of the ACM **9**, 11–12 (1962)
20. Zugan, D., Požar, R., Brodnik, A.: Floyd–warshall algorithm for sparse graphs. Algorithms **18**(12) (2025). https://doi.org/10.3390/a18120766, https://www.mdpi.com/1999-4893/18/12/766